

Ray tracing – Curved quadratic elements

1. Tetrahedron and ray

Surface position on the tetrahedron's face (weighted sum of shape functions):

$$\mathbf{P}(g, h) = \sum_{i=0}^5 N_i(g, h) \cdot \mathbf{f}_{\mathbf{nodes}_i} \quad (1)$$

Ray:

$$\mathbf{R}(t) = \mathbf{o} + t\mathbf{d} \quad (2)$$

where $\mathbf{d}$ is the ray's direction, and $\mathbf{o}$ is the ray's origin.

2. Intersection by minimising vector difference

2.1. Intersection problem

Face-ray intersection (3 unknowns) presented as the vector difference [1, 2]:

$$\mathbf{o} + t\mathbf{d} - \mathbf{P}(g, h) = \mathbf{0} \quad (3)$$

Nonlinear system of equations:

$$\mathbf{F}(t, g, h) = \mathbf{o} + t\mathbf{d} - \sum_{i=0}^5 N_i(g, h) \cdot \mathbf{f}_{\mathbf{nodes}_i} = \mathbf{0} \quad (4)$$

The system of equations contains 3 unknowns and 3 equations, so the mapping is:

$$\Re^3 \rightarrow \Re^3 \quad (5)$$

They represent the distance vector between a point on the ray and a point on the surface.

2.2. Möller-Trumbore algorithm for linear triangle intersection (initial guess)

$$\mathbf{E}_1 = V_1 - V_0, \mathbf{E}_2 = V_2 - V_0 \quad (6)$$

$$\mathbf{p} = \mathbf{d} \times \mathbf{E}_2 \quad (7)$$

$$\det = \mathbf{E}_1 \cdot \mathbf{p} \quad (8)$$

$$\mathbf{t}_{vec} = \mathbf{o} - V_0 \quad (9)$$

Where V_0, V_1, V_2 are triangle vertices.

$$u = \frac{\mathbf{t}_{vec} \cdot \mathbf{p}}{\det} \quad (10)$$

$$\mathbf{q} = \mathbf{t}_{vec} \times \mathbf{E}_1 \quad (11)$$

$$v = \frac{\mathbf{d} \cdot \mathbf{q}}{\det} \quad (12)$$

$$t = \frac{\mathbf{E}_2 \cdot \mathbf{q}}{\det} \quad (13)$$

2.3. Newton-Raphson method

Since there are 3 variables and 3 equations, Newton-Raphson method could be used to find the solution. Only Jacobians of F are required.

Jacobian of F :

$$\mathbf{J}_F = \left[\frac{\partial \mathbf{F}}{\partial f}, \frac{\partial \mathbf{F}}{\partial g}, \frac{\partial \mathbf{F}}{\partial h} \right] \quad (14)$$

$$\mathbf{J}_F = \left[\mathbf{d}, -\frac{\partial \mathbf{P}}{\partial g}, -\frac{\partial \mathbf{P}}{\partial h} \right] \quad (15)$$

$$\frac{\partial \mathbf{P}}{\partial g} = \sum_{i=0}^5 \frac{\partial N_i}{\partial g} \cdot \mathbf{f}_{nodes_i} \quad (16)$$

$$\frac{\partial \mathbf{P}}{\partial h} = \sum_{i=0}^5 \frac{\partial N_i}{\partial h} \cdot \mathbf{f}_{nodes_i} \quad (17)$$

The update by solving a linear system:

$$\Delta = [\Delta t, \Delta g, \Delta h]^T \quad (18)$$

$$\mathbf{J}_F \cdot \Delta = -\mathbf{F}(t, g, h) \quad (19)$$

$$\begin{bmatrix} d_x & -\frac{\partial P_x}{\partial g} & -\frac{\partial P_x}{\partial h} \\ d_y & -\frac{\partial P_y}{\partial g} & -\frac{\partial P_y}{\partial h} \\ d_z & -\frac{\partial P_z}{\partial g} & -\frac{\partial P_z}{\partial h} \end{bmatrix} \begin{bmatrix} \Delta t \\ \Delta g \\ \Delta h \end{bmatrix} = -(\mathbf{o} + t\mathbf{d} - \mathbf{P}(g, h)) \quad (20)$$

3. Intersection by minimising scalar squared distance

3.1. Intersection problem

Face-ray intersection could be presented as the scalar squared difference [3]:

$$F(g, h) = |\mathbf{V}|^2 - (\mathbf{V} \cdot \mathbf{d})^2 \quad (21)$$

where

$$\mathbf{V} = \mathbf{P}(g, h) - \mathbf{o} \quad (22)$$

and

$$t = \mathbf{V} \cdot \mathbf{d} = (\mathbf{P}(g, h) - \mathbf{o}) \cdot \mathbf{d} \quad (23)$$

The equation contains 2 unknowns, so the mapping is:

$$\Re^2 \rightarrow \Re^1 \quad (24)$$

They represent the squared perpendicular distance between the ray and surface (Figure 1).

If a local minimum of F is zero, then this corresponds to a point where the ray intersects the surface. Those points where F is a minimum, but where $F > 0$, indicate that the ray misses the surface by a finite distance. This is actually desirable in that the algorithm will still converge near "silhouette edges" of surface, but will give a non-zero minimum.

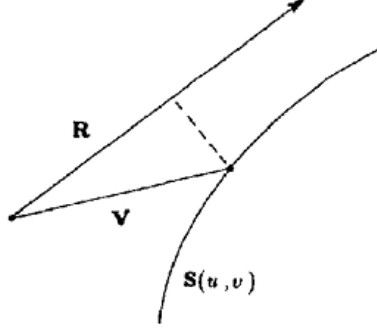


Figure 1: Distance from a point on the surface to a ray [3], S is P in this document.

3.2. Quasi-Newton method

It is not possible to use Newton-Raphson method in this case, as there are 3 variables and 1 equation. Additionally, this equation cannot be represented as a sum of squared residuals to be solved by Gauss-Newton approach. Consequently, it is necessary to know Jacobians as well as Hessians of F .

Jacobian of F :

$$\mathbf{J}_F = \left[\frac{\partial F}{\partial g}, \frac{\partial F}{\partial h} \right]^T \quad (25)$$

Let:

$$\mathbf{P}_g = \frac{\partial \mathbf{P}}{\partial g} \quad (26)$$

$$\mathbf{P}_h = \frac{\partial \mathbf{P}}{\partial h} \quad (27)$$

$$\mathbf{P}_{gg} = \frac{\partial^2 \mathbf{P}}{\partial g^2} \quad (28)$$

$$\mathbf{P}_{hh} = \frac{\partial^2 \mathbf{P}}{\partial h^2} \quad (29)$$

$$\mathbf{P}_{gh} = \mathbf{P}_{hg} = \frac{\partial^2 \mathbf{P}}{\partial gh} = \frac{\partial^2 \mathbf{P}}{\partial hg} \quad (30)$$

The partials of F are:

$$\frac{\partial F}{\partial g}(g, h) = 2 \left(\frac{\partial \mathbf{P}}{\partial g} \cdot \mathbf{V} - t \frac{\partial \mathbf{P}}{\partial g} \cdot \mathbf{d} \right) = \quad (31)$$

$$= 2 \frac{\partial \mathbf{P}}{\partial g} (\mathbf{V} - t\mathbf{d}) = \quad (32)$$

$$= 2 \frac{\partial \mathbf{P}}{\partial g} (\mathbf{P} - (\mathbf{o} + t\mathbf{d})) \quad (33)$$

$$\frac{\partial F}{\partial h}(g, h) = 2 \frac{\partial \mathbf{P}}{\partial h} (\mathbf{P} - (\mathbf{o} + t\mathbf{d})) \quad (34)$$

$$\frac{\partial F}{\partial g}(g, h) = 2 \mathbf{P}_g (\mathbf{V} - t\mathbf{d}) \quad (35)$$

$$\frac{\partial F}{\partial h}(g, h) = 2 \mathbf{P}_h (\mathbf{V} - t\mathbf{d}) \quad (36)$$

Hessian of F :

$$\mathbf{H}_F = \begin{bmatrix} \frac{\partial^2 F}{\partial g^2} & \frac{\partial^2 F}{\partial g \partial h} \\ \frac{\partial^2 F}{\partial h g} & \frac{\partial^2 F}{\partial h^2} \end{bmatrix} \quad (37)$$

The analytical formulae could be derived to calculate precise $\mathbf{H}_F$; however, the calculation process is quite time-consuming.

$$\frac{\partial F}{\partial g^2} = 2 [|\mathbf{P}_g|^2 - (\mathbf{P}_g \cdot \mathbf{d})^2 + \mathbf{V} \cdot \mathbf{P}_{gg} - (\mathbf{V} \cdot \mathbf{d})(\mathbf{P}_{gg} \cdot \mathbf{d})] \quad (38)$$

$$\frac{\partial F}{\partial h^2} = 2 [|\mathbf{P}_h|^2 - (\mathbf{P}_h \cdot \mathbf{d})^2 + \mathbf{V} \cdot \mathbf{P}_{hh} - (\mathbf{V} \cdot \mathbf{d})(\mathbf{P}_{hh} \cdot \mathbf{d})] \quad (39)$$

$$\frac{\partial F}{\partial g \partial h} = 2 [\mathbf{P}_g \cdot \mathbf{P}_h - (\mathbf{P}_g \cdot \mathbf{d})(\mathbf{P}_h \cdot \mathbf{d}) + \mathbf{V} \cdot \mathbf{P}_{gh} - (\mathbf{V} \cdot \mathbf{d})(\mathbf{P}_{gh} \cdot \mathbf{d})] \quad (40)$$

The update by solving a linear system:

$$\mathbf{x} = [g, h]^T \quad (41)$$

$$\Delta = [\Delta g, \Delta h]^T \quad (42)$$

$$\Delta = -s \mathbf{H}_F^{-1} \mathbf{J}_F \quad (43)$$

$$\mathbf{H}_F \cdot \Delta = -s \mathbf{J}_F \quad (44)$$

$$\begin{bmatrix} \frac{\partial^2 F}{\partial g^2} & \frac{\partial^2 F}{\partial g \partial h} \\ \frac{\partial^2 F}{\partial h g} & \frac{\partial^2 F}{\partial h^2} \end{bmatrix} \begin{bmatrix} \Delta g \\ \Delta h \end{bmatrix} = -s \begin{bmatrix} \frac{\partial F}{\partial g} & \frac{\partial F}{\partial h} \end{bmatrix}^T \quad (45)$$

where s is a scalar satisfying the following relationship:

$$s = \min_s (F(\mathbf{x} - s\mathbf{H}_F^{-1}\mathbf{J}_F)) \quad (46)$$

The scalar s can be found by employing backtracking line search with Armijo condition in the following direction [4]:

$$\mathbf{p} = -\mathbf{H}_F^{-1}\mathbf{J}_F \quad (47)$$

The directional derivative is:

$$D_p = \mathbf{J}_F \cdot \mathbf{p} \quad (48)$$

Iterate $\mathbf{x}$ as:

$$\mathbf{x} = \mathbf{x}_{init} + s\mathbf{p} \quad (49)$$

$$s = s\beta \quad (50)$$

Iterate until Armijo condition is fulfilled:

$$F(\mathbf{x}) \leq F(\mathbf{x}_{init}) + csD_p \quad (51)$$

4. Surface normal

$$\mathbf{n} = \frac{\partial \mathbf{P}}{\partial g} \times \frac{\partial \mathbf{P}}{\partial h} \quad (52)$$

References

- [1] A. Glassner, An Introduction to Ray Tracing, The Morgan Kaufmann Series in Computer Graphics, Morgan Kaufmann, 1989. URL: <https://books.google.co.uk/books?id=t3XNCgAAQBAJ>.

- [2] D. L. Toth, On ray tracing parametric surfaces, SIGGRAPH Comput. Graph. 19 (1985) 171–179. URL: <https://doi.org/10.1145/325165.325233>. doi:10.1145/325165.325233.
- [3] K. I. Joy, M. N. Bhetanabhotla, Ray tracing parametric surface patches utilizing numerical techniques and ray coherence, SIGGRAPH Comput. Graph. 20 (1986) 279–285. URL: <https://doi.org/10.1145/15886.15917>. doi:10.1145/15886.15917.
- [4] J. J. E. Dennis, R. B. Schnabel, Numerical Methods for Unconstrained Optimization and Nonlinear Equations, The Morgan Kaufmann Series in Computer Graphics, Society for Industrial and Applied Mathematics, 1996.